任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关？

以 DVWA 的用户 ID 查询功能为例：页面需要根据用户输入的 ID，从后台数据库中取出对应的用户信息并展示。比如输入 1，页面就会从 users 表中查询 ID 为 1 的用户姓名、头像等数据，再渲染到页面上。这类 “输入→取数据→返回结果” 的功能，都需要依赖数据库存储和查询数据，因此用户输入会直接关联数据库操作。

2. 一个最小业务链路是什么？

最小业务链路：用户输入条件 → 后端拼接 SQL 查询 → 数据库执行查询并返回结果 → 页面渲染结果

以用户 ID 查询为例：用户在输入框提交 ID1 → 后端拼接 SQL 语句 SELECT first_name, last_name FROM users WHERE user_id = '1' → 数据库执行查询并返回用户数据 → 页面显示对应的用户信息。

3. 用户输入是如何进入查询条件的？

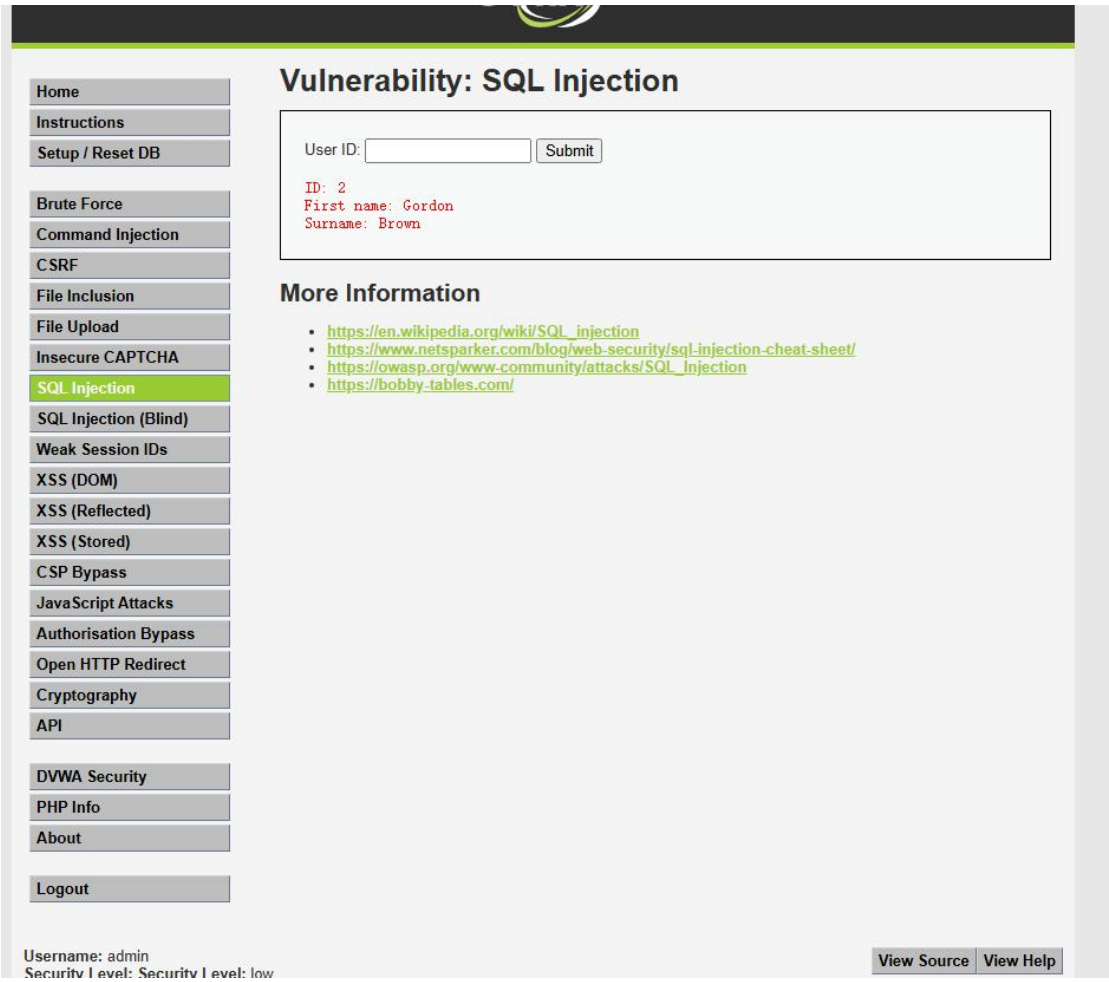
在 DVWA Low 级别的 SQL 注入场景中，后端代码没有对用户输入做任何过滤，直接将输入内容拼接到 SQL 语句中。比如用户输入的 ID 值，会被直接拼接到 WHERE user_id = '用户输入'这个条件里，用户输入的内容会完全成为 SQL 语句的一部分。

4. 为什么这为 SQL 注入创造了可能性？

因为后端直接将用户输入拼接到 SQL 语句中，用户可以输入恶意内容来修改 SQL 的原始逻辑。比如输入 1' OR '1'='1，拼接后的 SQL 会变成 SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1'，'1'='1'永远为真，数据库就会返回所有用户数据，原本的查询逻辑被完全篡改，这就是 SQL 注入的基础。

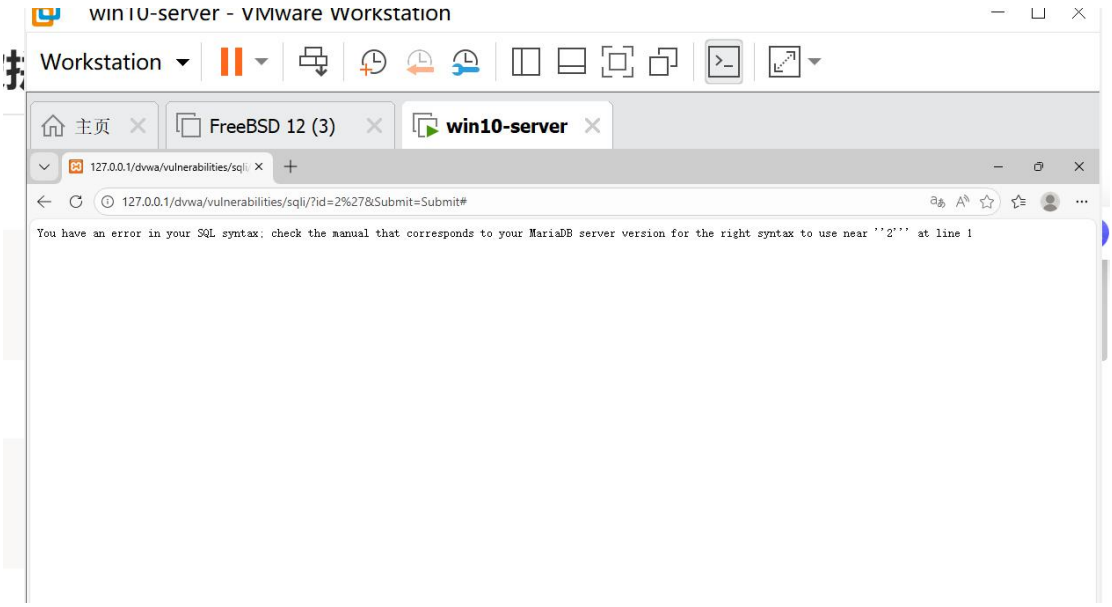
任务 2：DVWA SQL Injection Low 现象记录

普通输入 2



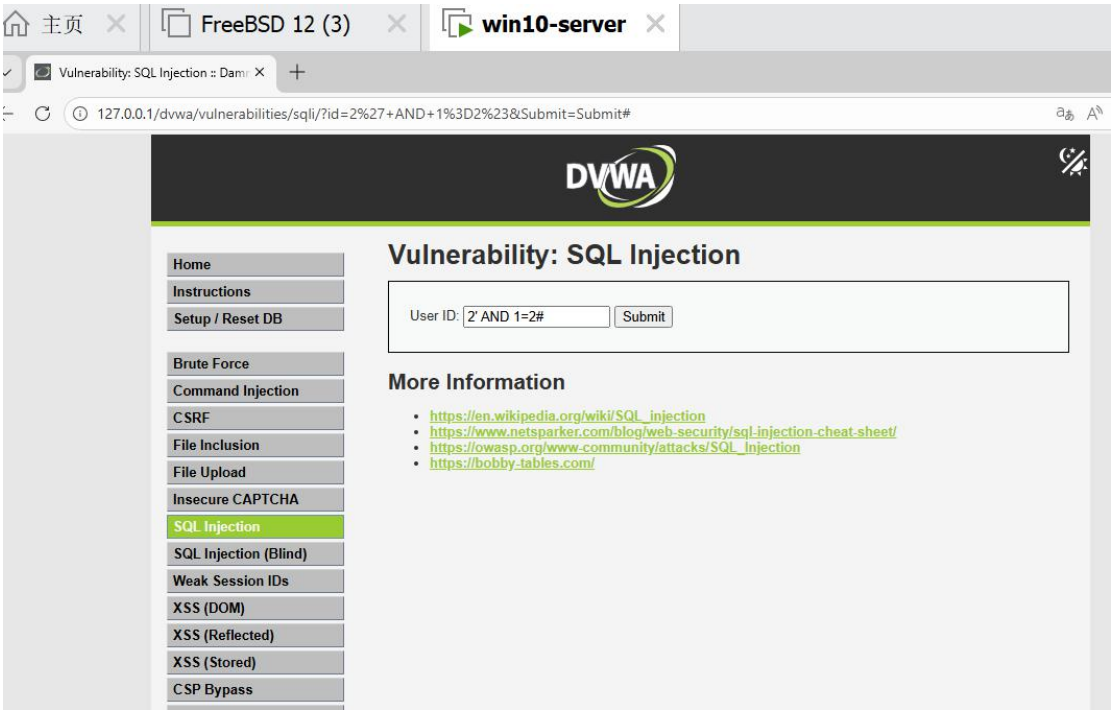
这是正常的业务查询，输入被当作 “查询条件”，按预期返回对应数据。

异常输入 2'



输入的单引号被直接拼接到 SQL 语句中，导致语法报错，证明后端完全没有过滤输入，直接拼接 SQL，这是注入的关键证据。

注入语句 2' AND 1=2#



输入被解析为 SQL 逻辑的一部分，AND 1=2 这个恒假条件生效，导致查询结果为空。这不是 “查不到用户”，而是用户输入篡改了 SQL 的查询逻辑，证明了注入漏洞的存在。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户输入通过 “字符串拼接” 的方式直接嵌入 SQL 语句中，完整链路为：用户输入 → 后端代码直接拼接字符串 → 形成完整 SQL 语句 → 数据库解析 SQL 语法并执行 → 执行结果返回后端 → 页面渲染结果比如输入 1' OR '1'='1，后端直接将输入拼接到 SQL 中，原本的查询结构被用户输入的 OR、'等字符修改，形成了攻击者可控的恶意 SQL 语句。

2. 为什么根因是 “结构被污染” 而不是 “字符本身危险”？

'、OR 这些字符本身没有危险性，它们只是 SQL 语法的一部分。问题的核心是后端将用户输入直接拼接到了 SQL 语句的结构中，让用户输入有能力修改 SQL 的语法逻辑。如果使用参数化查询，即使用户输入包含' OR '1'='1，它也只会被当作一个普通的字符串参数，不会被数据库解析为 SQL 语法的一部分，也就无法修改查询结构。因此，危险的不是字符本身，而是 “输入被嵌入查询结构，导致结构被污染”。

3. Low 与 Medium 的差异在哪里？

Low 级别：无任何输入过滤，直接将用户输入拼接到 SQL 语句中，且使用字符串类型参数（带单引号），可以直接通过'闭合单引号来篡改 SQL 结构，同时页面会直接暴露数据库错误信息。

Medium 级别：对用户输入做了简单过滤，用 mysql_real_escape_string()转义了单引号、双引号等特殊字符，同时将参数改为数字类型（不带单引号），无法直接用'闭合字符串；但它只是过滤了字符，没有解决 “拼接 SQL” 的核心问题，依然可以通过其他方式（如联合查询、数字型注入）篡改查询结构，只是注入难度有所提升。

4. 普通 SQL 注入与 Blind SQL Injection（盲注）的差异在哪里？

普通 SQL 注入：页面会直接返回 SQL 查询的结果或错误信息，攻击者可以直接从页面中

拿到数据或判断 SQL 执行结果，比如 DVWA Low 级别中，输入注入语句后直接返回所有用户数据，或输入错误单引号直接暴露语法错误。
盲注：页面不会直接返回数据库数据或错误信息，只会返回 “查询成功 / 失败” 两种状态（如显示正常页面或提示 “找不到用户”），攻击者需要通过页面的响应差异（如响应时间、页面内容变化）来推断数据库的信息，比如通过 AND SLEEP(5)判断语句是否被执行，再逐位猜解数据。

5. 为什么盲注和普通注入根因相同，但证据形式不同？

根因相同：两者的核心都是查询结构被污染，攻击者可以控制 SQL 语句的执行逻辑，本质都是拼接 SQL 导致的注入漏洞。
证据形式不同：普通注入可以直接通过页面返回的错误信息、查询结果看到注入效果，证据是直接的；而盲注没有直接的结果或错误信息，只能通过页面的间接响应（如延时、内容变化）来推断注入是否成功，证据是间接的。

强制判断题

SQL 注入的根因更接近：**B. 输入进入了不该进入的查询结构**
理由：SQL 注入的核心是用户输入被当作 SQL 语法的一部分解析，修改了查询的逻辑结构，而不是输入里的字符本身有危险。即使没有特殊字符，只要能污染查询结构，依然可能存在注入漏洞。
Blind SQL Injection 和普通 SQL 注入的最大差异更接近：**B. 证据形式不同**
理由：两者的根因都是 SQL 语句被拼接、查询结构被污染，漏洞的本质没有区别；差异仅在于页面是否直接返回数据或错误信息，导致攻击者获取证据的方式不同。

任务 4：SQL 注入防护清单与解释

防护措施	解决的问题	控制的层级	它不能替代什么
参数化查询（预编译语句）	从根本上解决 SQL 注入问题，让用户输入永远作为 “数据” 而非 “SQL 语法” 被处理，无法污染查询结构	**根因控制层**（直接解决拼接 SQL 的核心问题）	不能替代输入校验、权限控制等其他防护，也无法防御业务逻辑层面的漏洞
输入校验	限制用户输入的格式、长度、内容（如 ID 仅允许数字），减少恶意输入的可能性	**暴露面控制层**（缩小可被利用的输入范围）	不能替代参数化查询，复杂场景下的输入校验容易被绕过，无法完全防止注入
错误处理	隐藏数据库的详细错误信息，防止攻击者通过错误信息推断数据库结构	**暴露面 / 证据隐藏层**（减少攻击者可利用的信息）	不能解决注入漏洞本身，只是让攻击者更难判断漏洞是否存在，无法阻止漏洞被利用
最小权限原则	给应用程序使用的数据库账号分配最低必要权限（如仅允许 SELECT，禁止 DROP、ALTER 等高危操作）	**后果缩减层**（降低漏洞被利用后的影响范围）	不能阻止注入漏洞被触发，只能限制攻击者能执行的操作，无法从根源解决问题
使用 ORM 框架	框架自动处理 SQL 语句，避免开发者手动拼接 SQL，强制	**开发层 / 根因辅助控制**	不能替代参数化查询的核心防护，ORM 框架

防护措施	解决的问题	控制的层级	它不能替代什么
	使用参数化查询	（减少拼接 SQL 的场景）	若使用不当（如原生 SQL 拼接）依然可能存在注入

收口项 1：SQL 注入因果链 / 防护关系表

阶段	核心行为	风险点	对应防护控制点
输入点	用户提交输入（如 ID、搜索内容）	恶意输入被提交	输入校验（限制输入格式 / 内容）
查询构造	后端将用户输入直接拼接为 SQL 语句	输入污染 SQL 结构，形成恶意 SQL	参数化查询（预编译语句）
数据库解析 / 执行	数据库解析并执行 SQL 语句	恶意 SQL 被执行，数据被泄露 / 篡改	最小权限原则（限制执行权限）
结果差异 / 行为证据	页面返回查询结果或错误信息	攻击者获取漏洞证据	错误处理（隐藏详细错误信息）
根因判断	核心根因：SQL 语句拼接导致输入污染查询结构	–	–

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？

我最清楚的是 SQL 注入的核心根因是 “SQL 语句被拼接，用户输入污染了查询结构”，而不是输入中的特殊字符本身有危险，理解了参数化查询为什么能从根源解决注入问题。

2. 我最容易混淆的一点是什么？

最容易混淆的是 “不同防护措施对应的控制层级”，比如一开始容易把输入校验和参数化查询的作用搞混，分不清哪个是解决根因，哪个是缩小暴露面。

3. **我本次作业中哪一段最需要教师复核**？

对 Low 与 Medium 级别注入差异的分析，以及盲注和普通注入的证据形式差异部分，担心自己对两者的区别理解不够准确。

4. 如果重做一次，我会优先改哪一部分？

会优先完善防护清单部分，补充更多实际开发中的防护措施（如 ORM 框架、WAF 的作用），并更清晰地区分每个措施的控制层级。

收口项 3：AI 输出审核与人工复核记录

1.本作业哪些内容使用过 AI 辅助？

SQL 注入的基础概念梳理、作业结构框架搭建

部分防护措施的层级划分和解释辅助

收口项自评表的内容参考

2.哪些判断是你自己复核后保留的？

SQL 注入根因是 “结构被污染” 而非 “字符危险” 的核心判断

普通注入与盲注的根因相同、证据形式不同的判断

各防护措施对应的控制层级划分

3.哪些内容你发现 AI 输出不准确并进行了修正？

修正了 AI 输出中 “输入校验可以完全防止 SQL 注入” 的错误表述，明确输入校验只能缩小暴露面，无法替代参数化查询

修正了 AI 对 DVWA Low 级别 SQL 语句的拼接示例，确保与实际 DVWA 源码逻辑一致

4.你最终如何确认结论和证据是一一对应的？

对照 DVWA Low 级别的实际实验现象，验证了输入不同内容时页面的返回结果，确认了普通输入、异常输入的差异与分析一致

结合 DVWA 源码逻辑，核对了 SQL 语句拼接的过程，确认了 “结构被污染” 的根因分析与实际代码逻辑匹配

对照作业要求，逐条检查每个任务的要点是否覆盖，确保结论符合作业的最低完成要求